

Algoritmos y Estructuras de Datos III

Trabajo Práctico N° 1	Unidad 1
Modalidad: Semi -Presencial	Estratégica Didáctica: Trabajo individual
Metodología de Desarrollo: Det. docente	Metodología de Corrección: Vía Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	Fecha Entrega: A confirmar por el Docente.

MARCO TEÓRICO (COMPLEJIDAD ALGORÍTMICA):

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

Parte I – Introducción a la Materia.

1. Describir el Concepto de Complejidad en el tiempo

La complejidad en el tiempo, que es la cantidad de tiempo que un algoritmo utiliza cuando aumentan los datos de entrada, se hace una estimación de cómo realizaría un algoritmo su trabajo independientemente del tipo de máquina where would run. Se obtiene "contando" cuántas operaciones realiza el código y se expresa como una función del tamaño de entrada mediante notación Big-O; el algoritmo que crece menos en el tiempo es el más rápido.

2. Explicar el concepto del Ave, Cohete e Internet.

Son una alegoría donde se ejemplifican distintos comportamientos de la complejidad temporal en el envío de archivos:

Ave: el tiempo es constante - $O(1)$. Por muy grande que sean los archivos, el ave siempre tardará lo mismo en llegar. El tiempo no varía con el tamaño de la entrada.

Internet: el tiempo es lineal - $O(n)$. El tiempo de llegada depende del tamaño del archivo y la velocidad de conexión. Cuanto más grande es el tamaño, el tiempo también lo es: este tiempo sí varía de forma proporcional.

Cohete: el tiempo es cuadrático o crece muy rápidamente - $O(n^2)$. El tiempo varía, y lo hace rápidamente cuando aumenta el tamaño de los datos. Es el peor de todos en términos de escalabilidad.

3. Que se entiende por notación asintótica, para que sirve

La notación asintótica es un tipo de medida de la eficiencia que se emplea cuando se considera que se ignoran algunos términos siguiendo el mismo razonamiento que se estaba utilizando previamente, que lleva a la proyección de la eficiencia del algoritmo, o, por el contrario, cuando el cálculo de la función es extremadamente difícil o, incluso, imposible y nos podemos quedar con aproximaciones a la función de eficiencia.

Las notaciones asintóticas son representaciones que se utilizan en matemáticas para mostrar la complejidad del tiempo de los mismos. Sirven para comparar la eficiencia en la ejecución de cada uno de estos algoritmos. Estos sirven para visualizar cuánto le cuesta a un algoritmo una entrada, en función del tamaño de la entrada; no es que se mida el tiempo de ejecución real, sino, simplemente, que se busca ver cómo (o de qué manera) aumenta, a medida que aumenta la entrada, el tiempo (o espacio) de ejecución. Por lo tanto, permite tener una medida que no depende de constantes que dependen de la máquina utilizada para implementarlos y probarlos, pues no se necesita implementar cada uno de estos algoritmos y ejecutarlos para hacer la comparación que se desea.

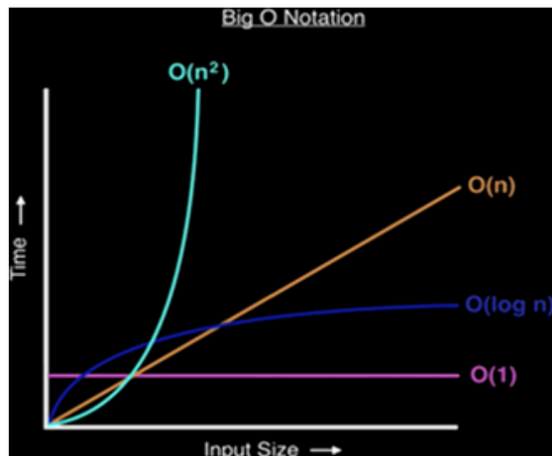
4. Describir, Big O, Big Omega y Big Theta

Big O: Especifica un límite superior para una función, limitando únicamente una función desde encima de ella. Acota el peor de los casos, es decir, el mayor tiempo en la ejecución del programa.

Big Omega: Ofrece un límite inferior asintótico. Puede ser interesante cuando se tiene un límite inferior en la complejidad temporal de un algoritmo. Es la notación menos utilizada de las tres, ya que el mejor rendimiento de un algoritmo generalmente no resulta útil.

Big Theta: Limita una función tanto de forma superior como de forma inferior, describiendo por lo tanto el comportamiento asintótico correcto.

5. Explicar la siguiente Gráfica de Big O



La grafica de Big O Notation muestra el Tiempo (Time ->) en el eje vertical y el Tamaño de Entrada (Input Size ->) en el eje horizontal. Es una gráfica que muestra cuatro curvas con distintas complejidades:

$O(1)$ - Tiempo constante: línea horizontal. No importa cuánto crezca la entrada, el tiempo de ejecución es siempre el mismo. La mejor complejidad que se podría tener.

$O(\log n)$ - Tiempo logarítmico: curva que crece muy lentamente. Para n duplicado el tiempo aumenta solo en una unidad. Por ejemplo: búsqueda binaria.

$O(n)$ - Tiempo lineal: línea diagonal. El tiempo crece mediante un incremento proporcional al tamaño de la entrada. Ejemplo: búsqueda simple.

$O(n^2)$ - Tiempo cuadrático: curva que va subiendo con mucha pronunciación, el tiempo crece exponencialmente al crecer n . Por ejemplo, bubble sort, ordenación mediante selección.

Las complejidades $O(1)$ y $O(\log n)$ están dentro de la "Good Zone" conocida como zona buena, en cambio $O(n^2)$ está dentro de la "Bad Zone" que representa la zona mala. El algoritmo que crece menos en el tiempo es más rápido.

6. Explicar la diferencia entre Complejidad en el Tiempo y Complejidad Espacial.

La Complejidad del Tiempo y la Complejidad del Espacio son dos algoritmos que permiten evaluar la complejidad de un proceso. La primera mide el tiempo que puede tardar un algoritmo en computar hasta que pueda finalizar, y la segunda se refiere a cuánta memoria se debe asignar para que el algoritmo pueda ejecutarse correctamente. Ambas métricas son de carácter teórico y describen cuál va a ser el comportamiento asintótico del algoritmo en función de una cantidad de recursos que se escalará con la magnitud y el número de elementos del conjunto de datos que se consideren.

A diferencia de los tiempos que se ocupan en cumplir las órdenes ejecutadas, para el cálculo de la complejidad espacial hay que sumar el tiempo adicional + el espacio asociado por los valores de entrada. En consecuencia, la memoria que debe ser consumida depende de la cantidad de datos que se manejen.

En el proceso de creación de software, la complejidad del tiempo es normalmente prioritaria. Lo que sí es cierto es que el rendimiento (velocidad) de un software de aplicación es crítico. Aun así, se sigue la estricta medida del límite de la memoria física de las plataformas. Ignorar la complejidad del espacio puede ser un error catastrófico, ya que, nunca se debe ejecutar una función que consuma más espacio que el que tenemos disponible en el sistema, ya que acabaría en un fallo en la ejecución.

MARCO PRÁCTICO

1. Determinar y crear la Función Big O para los siguientes Códigos:

Complejidad algorítmica

Indicar, con notación "Big-O", la complejidad algorítmica (respecto al tiempo) de cada fragmento de código mostrado a continuación.

```
for i = 0 to N:  
    if i % 2 == 0:  
        print i
```

1

```
for i = 0 to N:  
    for j = i to N:  
        print i + "," + j
```

3

```
for i = 0 to length(A):  
    for j = 0 to length(B):  
        print A[i] + "," + B[j]
```

4

```
for i = 0 to N:  
    j = 1  
    while j < N:  
        print j  
        j = j * 2
```

6

```
for i = 0 to N:  
    if i % 2 == 0:  
        print i  
for i = 0 to N:  
    if i % 2 != 0:  
        print i
```

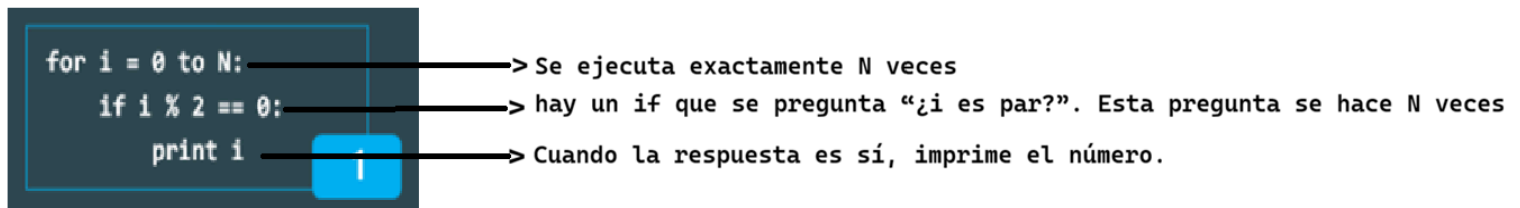
2

```
i = 1  
while (i < N):  
    print i  
    i = i * 2
```

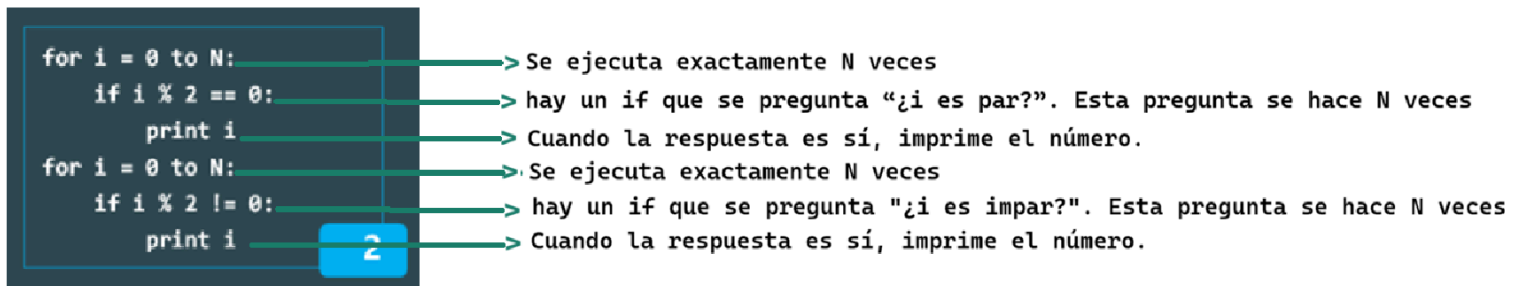
3

```
N = 1000  
if N % 2 == 0:  
    print "par"  
else:  
    print "impar"
```

6

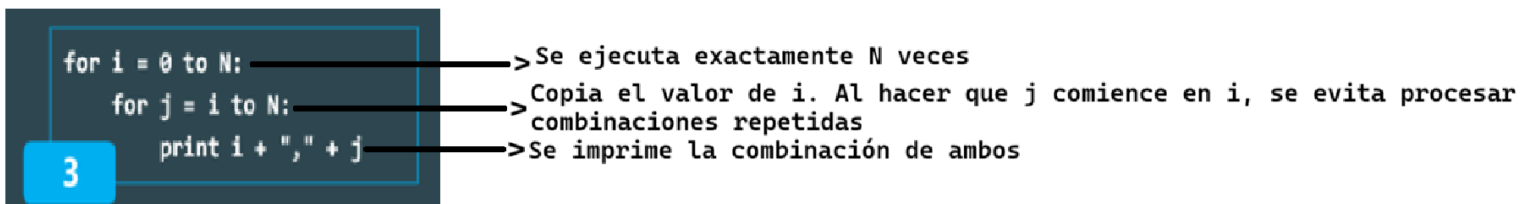


Se tiene que recorrer todo el bucle de 0 a N. Por eso, el tiempo crece de forma proporcional a N.
Este código es $O(N)$ - Tiempo lineal.

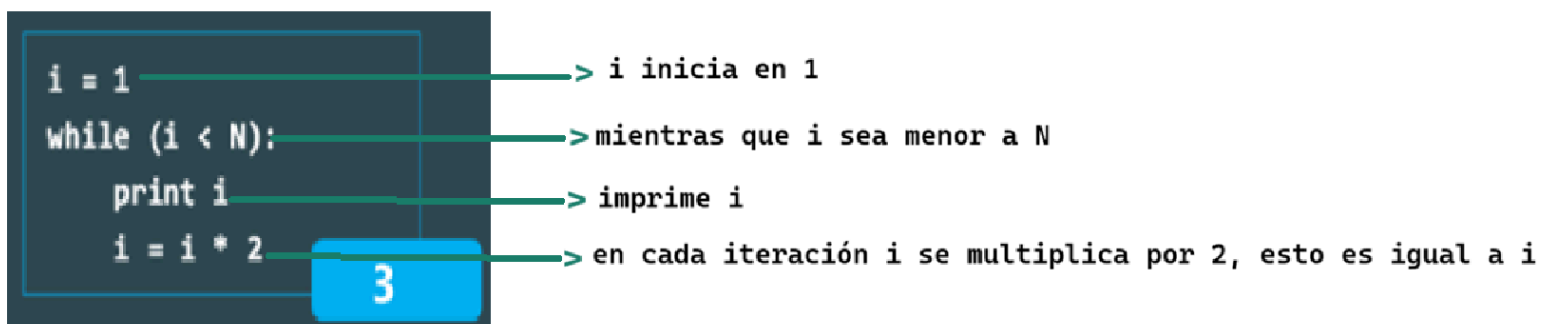


Aunque solo imprime los números pares, la mitad de las veces, y la otra mitad los impares se va a tener que recorrer todo el bucle de 0 a N.

Como el crecimiento es proporcional a N. Este código es $O(N)$ - Tiempo lineal



Aunque no llega a $N \times N$ completo, sigue creciendo de forma cuadrática.
Este código es $O(N^2)$ - Tiempo cuadrático



En cada iteración se duplica el resultado.
Este código es $O(\log n)$ - Tiempo logarítmico

```
for i = 0 to length(A):  
  for j = 0 to length(B):  
    print A[i] + "," + B[j]
```

4

- > El for i inicia en 0 y recorre todos los elementos del arreglo A.
- > El for j inicia en 0 y recorre todos los elementos del arreglo B. Para cada A.
- > Por eso el total es $O(a * b)$. La cantidad total de operaciones depende del tamaño de ambos arreglo

Hay dos bucles anidados iterando sobre arreglos distintos A y B
Este código es $O(n^2)$ - Tiempo cuadrático.

```
N = 1000  
if N % 2 == 0:  
  print "par"  
else:  
  print "impar"
```

6

- > N inicia en 1000
- > Se divide N por 2. Como el resto es 0, la condición es verdadera.
- > imprime par
- >
- > si la condición es distinta de 0, imprime impar

Al ser la ejecución igual. Este código es $O(1)$ - Tiempo constante

```
for i = 0 to N:  
  j = 1  
  while j < N:  
    print j  
    j = j * 2
```

6

- > inicia en $i = 0$, y recorre todo según el valor de $N + 1$
- > j inicia en 1
- > mientras que j sea menor a N
- > imprime j
- > en cada iteración de j se lo multiplica por 2 dando $j = j * 2$

En cada iteración se duplica el resultado.
Este código es $O(\log n)$ - Tiempo logarítmico
Este código en el total es Big O: $O(n \log n)$

La Cátedra.
Lic. Oemig José Luis.